本形學
Ontypology

- 從異同到分合，從分合到形，從形到理解這條路，大腦走了億萬年。
- 邏輯學百年前是哲學，當看清楚後是顯學(科學)
- 今探討「形」(大腦概念) 是哲學，本形學將「形」，看清楚轉成顯學(科學)，並實践化 Mind AI 。

本形学 Ontypology ★

异同 → 分合 → 形 → 理解 → 显学(科学)



《IC 設計大腦》準確使用完全方案

Type-Driven Design Flow：從規格到 RTL 的雙本形驅動精密藍圖

SpecType

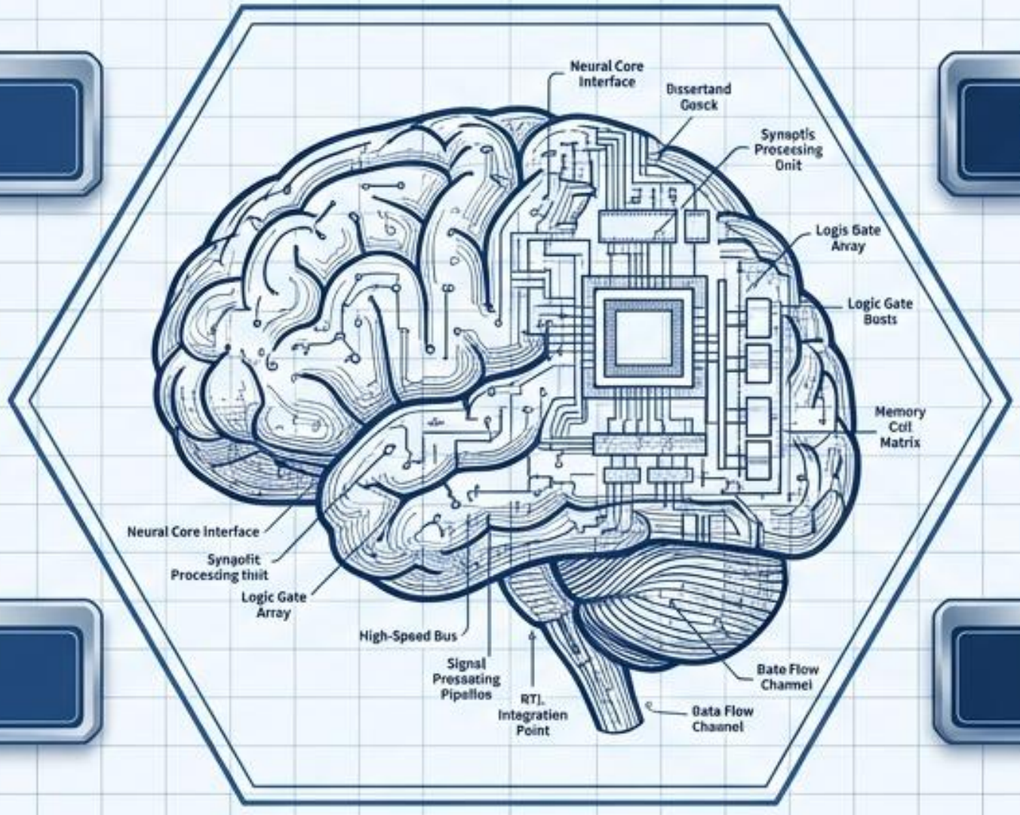
InterfaceType

ModuleType

TimingType

VerifyType

PatternType



不談抽象理論，只給可驗證、可回溯、零模糊的精確操作法。

Gemini Notebook

運作基石 —— 四條絕對契約

❶ 理論不重述

本形學（空間/時間本形）為已知操作語彙，不在此重述理論本體，直接落地應用於設計現場。

❷ RTL 為唯一準則

交付物嚴格限定為 Verilog/SystemVerilog 程式碼與型化證據；系統工具鏈或平台實作不在本方案範圍內。

❸ 譯名規約

英文一律使用 Type。

嚴格禁用 Form

違者當場更正，絕不妥協。

❹ 全程以「形」說話

任何產出必須能命名、附上 Ω(T) 判準，並留下型化證據。

無證據 = 未完成

Gemini Notebook

全域架構 —— 兩翼一支點

[空間本形 | Space]

確保「結構正確」

核心提問：這東西由什麼組成？



防堵錯誤：
- ☑ 部件遺漏
- ☑ 介面破洞
- ☑ 定義模糊

工具：BOM/SOM, 屬性表, ∩(T) 判準

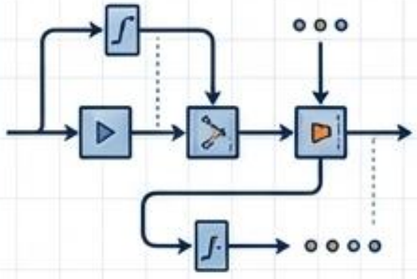
Mind AI 平台

語言歸納交給 LLM，
演繹智慧交給
Mind Model

[時間本形 | Time]

確保「過程正確」

核心提問：這結論怎麼推導出來的？



防堵錯誤：
- ☑ 推理跳步
- ☑ 無來源推理
- ☑ 經驗流失

工具：模式萃取, 演繹推理, 自我演化

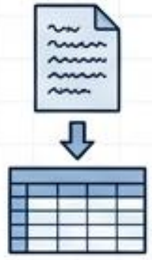
形庫 (Type Library)

沉澱雙翼共用的形與模式

Gemini Notebook

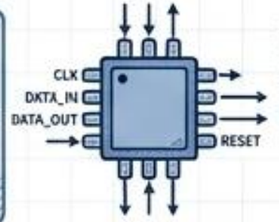
共同語言 —— IC 領域的六大「形」矩陣

[SpecType]
規格形
把模糊需求收容成可判定的形。



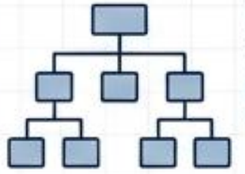
產出：屬性-屬性值表

[InterfaceType]
介面形
定義模組對外的訊號與時序契約。



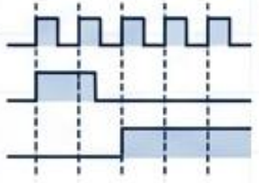
產出：表頭即形的訊號表

[ModuleType]
模組形
定義內部組成與空間依存關係。



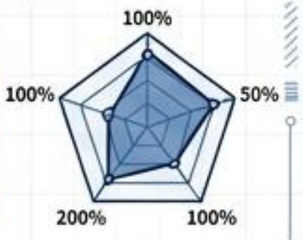
產出：BOM/SOM 結構

[TimingType]
時序形
定義時間上的行為契約與狀態切分。



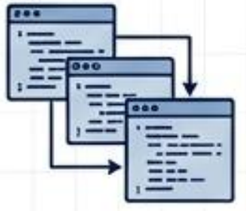
產出：狀態形、波形契約

[VerifyType]
驗證形
定義如何取證與判定設計正確性。



產出：Testbench、TypeCov

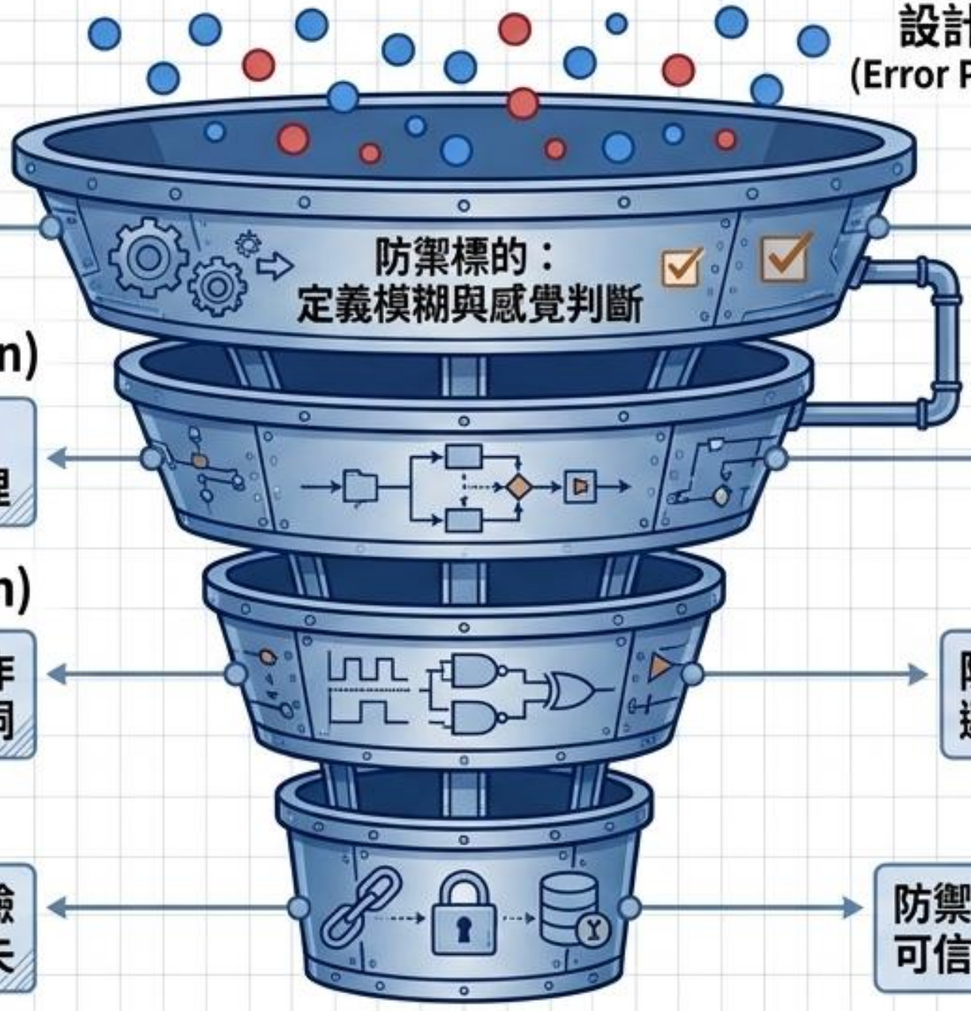
[PatternType]
模式形
沉澱可重複套用的最佳編碼模式。



產出：形庫模式卡

Gemini Notebook

防線建構 —— 四層準確性保證



① ∩(T) 交集判準

把關問題：
算不算這個形？

機制： 特徵集合逐項判定，
判出即退回重做。

② 演繹骨架 (Deductive Skeleton)

把關問題：結
論從哪裡來？

防禦標的：跳躍
推導與無來源推理

機制： 從判準嚴格演繹結構，
無來源結論一律刪除。

③ 形式驗證 (Formal Verification)

把關問題：波形
真的符合契約嗎？

防禦標的：實作
錯誤與邏輯漏洞

防禦標的：
邏輯漏洞

機制：理想模型比對、
反例清空。

④ 形證據鏈 (Evidence Chain)

把關問題：
經驗留存了嗎？

防禦標的：經驗
斷裂與資產流失

防禦標的：
可信嗎？

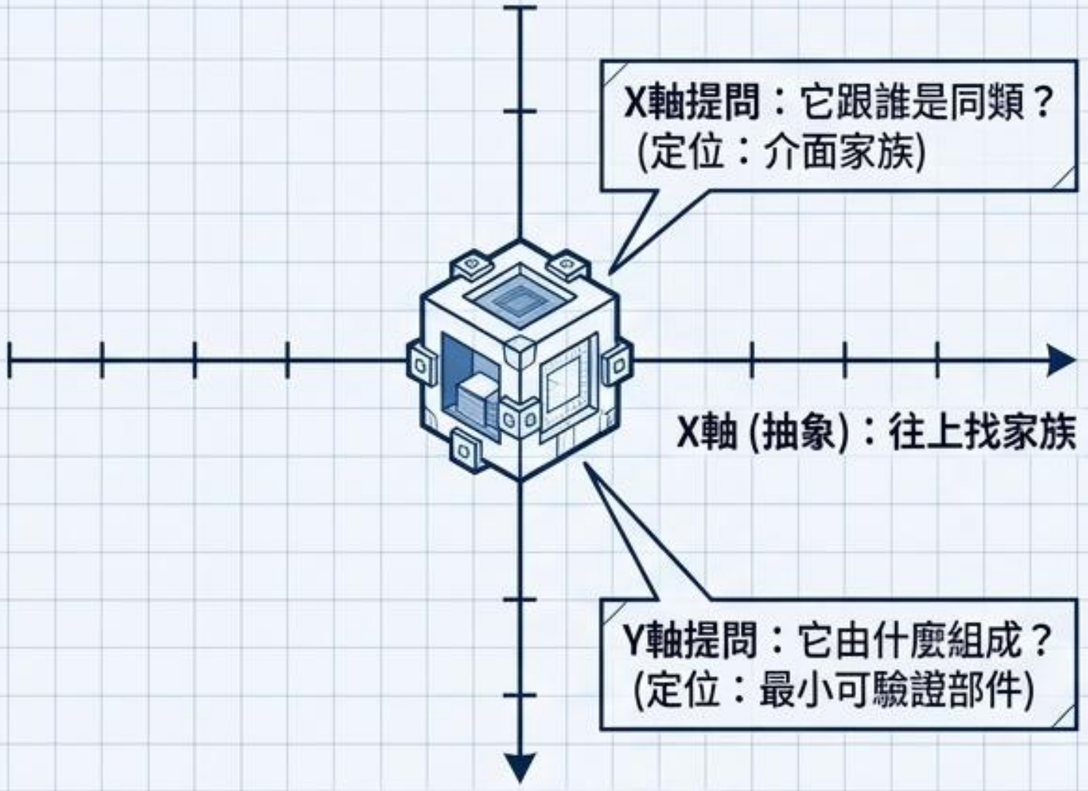
機制：每步留型化證據
證據斷裂視同未完成。

⚠ 越早的防線越便宜：第一層的一句精確判準，勝過第三層的三天除錯。

Gemini Notebook

空間原則大解密 —— 雙軸與三括號

雙軸定位網格



⚠ 評審設計必須雙軸提問，單軸思考視同不合格。

語意精確化的三括號



參數容器 (Parameter Container)

內容物可替換，模組身份不變 (例如：DATA_WIDTH)。



組成集合 (Composition Set)

BOM/SOM 結構。部件位置與依存固定，拿掉必要件整體塌陷。



抽象集合 (Abstraction Set)

IP 家族。不由清單列舉，而是由 ⋂(T) 判準動態生成。

Gemini Notebook

時間原則大解密 —— 嚴禁自由生成

✓ 正確路徑：演繹與拼裝

1. **∩(T) 判準**
   (明確的特徵定義)
2. **Mind Model 演繹骨架**
   (推導出必備結構)
3. **PatternType 模式選定**
   (選用預先驗證的積木)
4. **LLM 實體化**
   (輸出 Verilog RTL)

核心理念：程式碼只是模式的物理實體。
語言能力交給 LLM，結構決定權交給演繹骨架。

✗ 致命誤用：自由生成

規格文本
(Raw Specs)

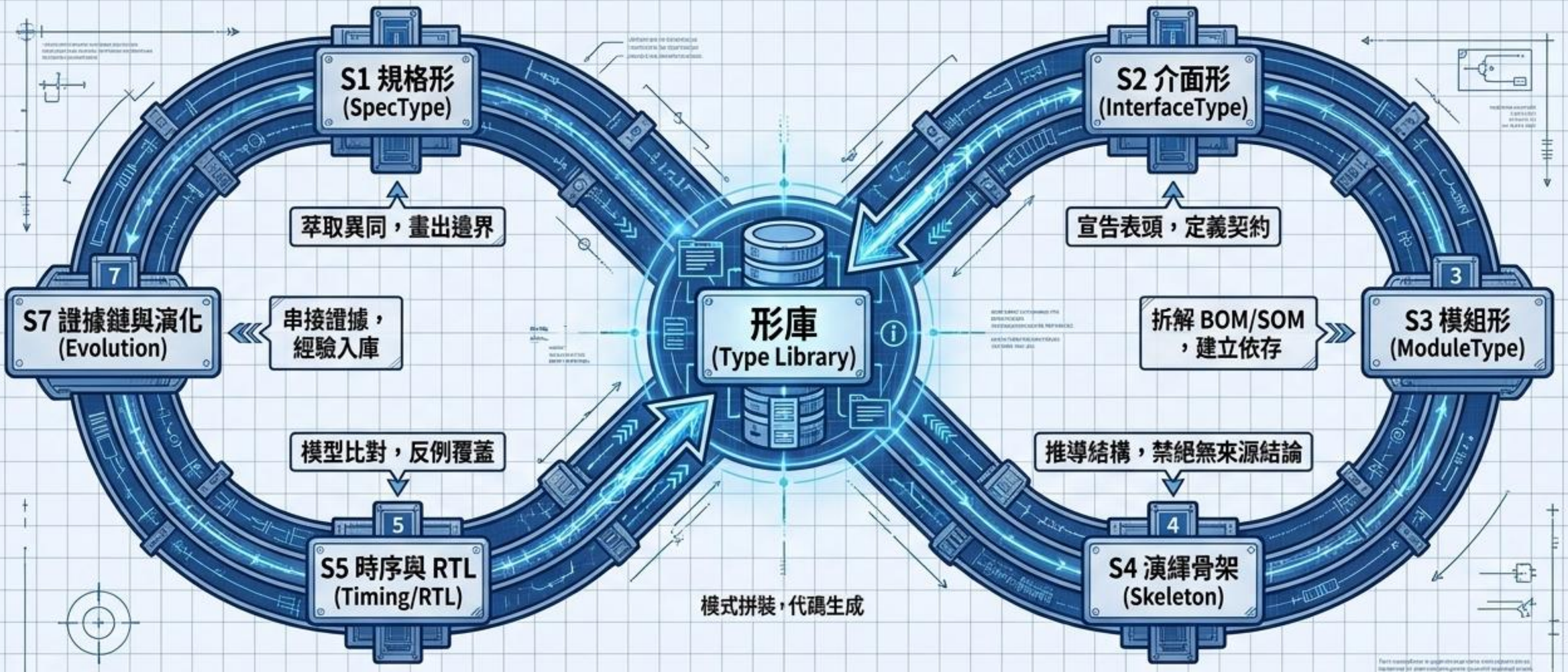
SYSTEM FAILURE

RTL 程式碼
(Code)

致命後果：沒有骨架的推理無法回溯。
出錯時無法定位是規格錯、骨架錯還是程式錯。
任何跳過骨架的產出，一律拒收！

Gemini Notebook

實戰引擎 —— 雙本形驅動 7 步驟總覽



上一步的輸出就是下一步的輸入。專案結束 = 形庫長大，經驗沉澱供下一次 S1 搜尋。

Gemini Notebook

深潛 S1-S2 —— 畫出邊界與介面契約

S1: 異同 ⇨ 分合 ⇨ 形

個案A : 系統時鍾為100MHz，數據寬度32位，需同步復位。

個案B : 系統時鐘為200MHz，數據寬度64位，需異步復位，增加校驗位。

屬性表

	維度 1 (時鐘)	維度 2 (數據寬度)	新維度 (復位/校驗)
個案 A	100MHz	32位	同步復位
個案 B	200MHz	64位	異步復位 / +校驗位

嚴格規則：一旦開新維度，必須回填所有既有個體。
屬性留白是結構漏洞！

S2 : 表頭即形

InterfaceType 訊號表

signal_name	direction	width	domain
clk	input	1	sys_clk
data	output	(DATA_WIDTH)	sys_clk

The Form/Type

The Instances

( ) 參數化替換值

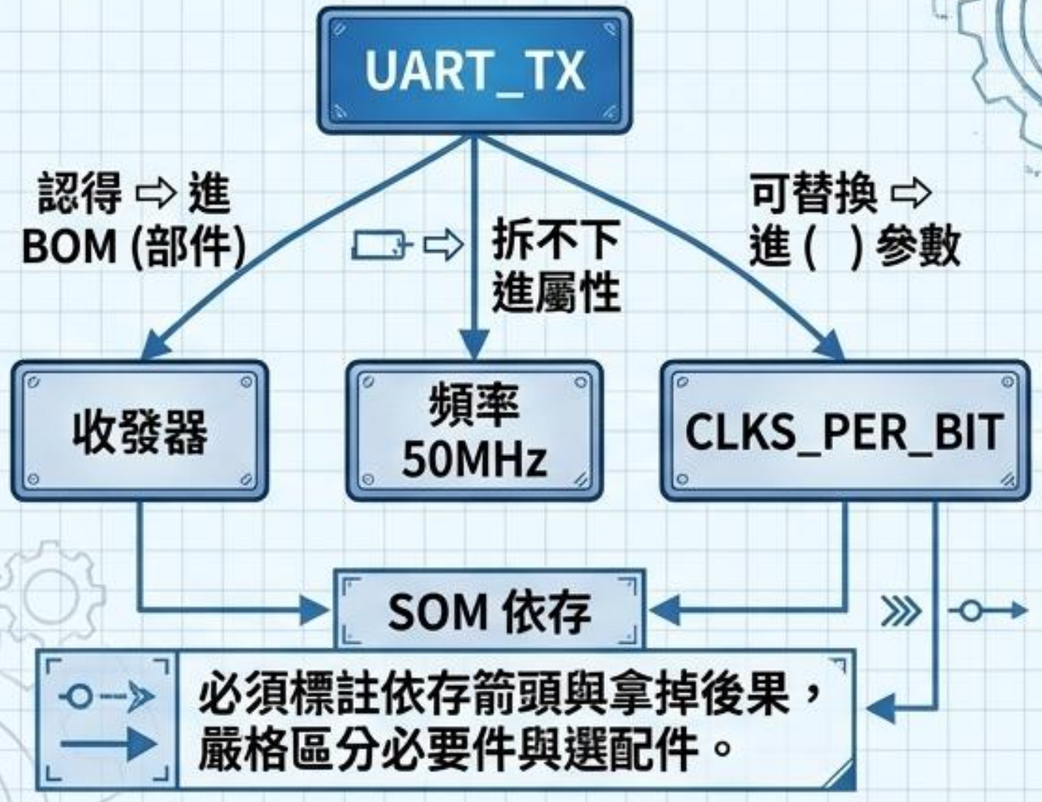
避坑指南：訊號表不是文件格式，它就是介面的「形」。
每一欄是屬性，每一列是實體。絕不為單一案例定義獨立表。

Gemini Notebook

深潛 S3-S4 —— 拆解模組與鑄造骨架

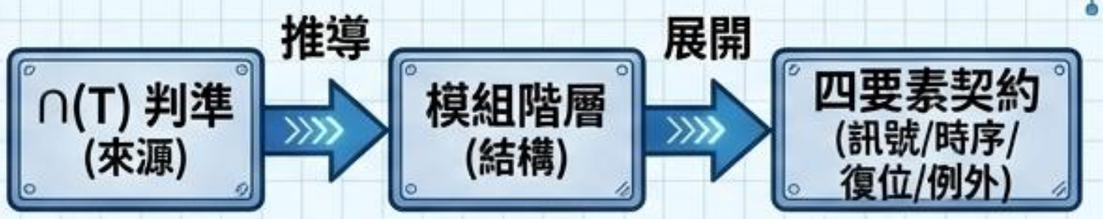
S3: 模組形分解

S3：拆下來後，它還認得自己嗎？



S4: 演繹骨架

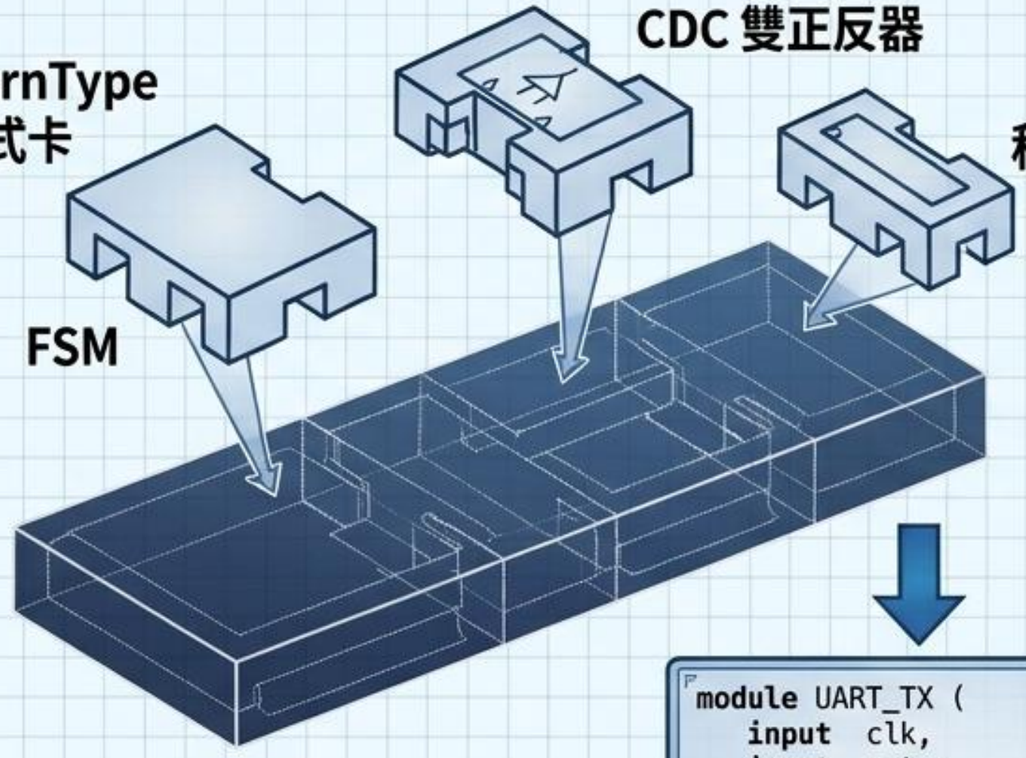
S4：讓結論有根有據



紅色警示：無來源的骨架結論，一律當場刪除。嚴禁依賴『憑經驗』的模糊說詞。

Gemini Notebook

深潛 S5 —— 時序形與模式拼裝



演繹骨架
(Deductive Skeleton)

```
module UART_TX (
    input  clk,
    input  rst_n,
    input  tx_start,
    input  [7:0] tx_data,
    output tx_busy,
    output tx_line
);
  // RTL CODE HERE
  // Pattern-based assembly
endmodule
```

設計順序：模式先於程式碼

程式碼只是模式的物理實體。先從形庫選定 PatternType，再進行拼裝。

時序形 (TimingType) 切分



將時間切分為明確的狀態形 (如：IDLE / START / DATA / STOP)。時間屬性作為容器的刻度。

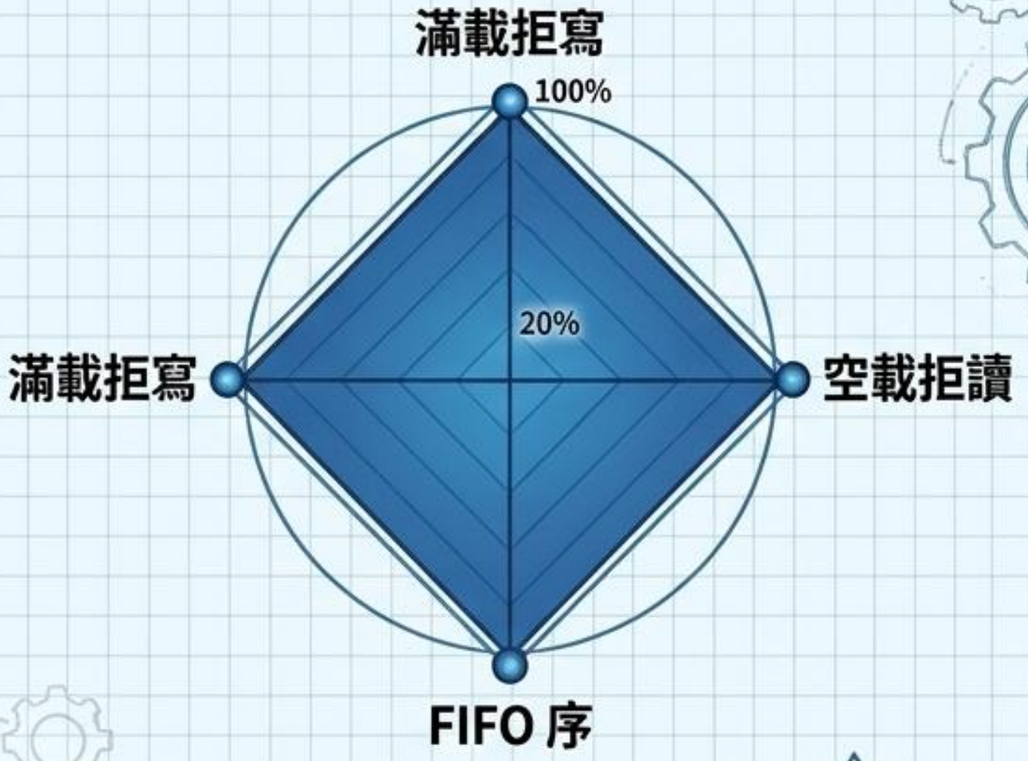
RTL 生成紀律

- 埠必須與 S2 訊號表逐欄對應。
- 註解必須標齊形 ID 與模式 ID，確保 100% 可回溯。

Gemini Notebook

深潛 S6 —— 驗證形：模型比對與誤注入

TypeCov 形覆蓋率



分母直接來自 ∩(T) 的可觀察條件。不追求虛胖的隨機測試數量，只對著判準收斂。

取證兩大支柱

1. 理想模型比對



2. 誤注入



避坑指南：拒絕只跑 Happy Path。反例必須被強制觸發，且必須解釋根因並標註（規格錯/骨架錯/RTL錯）。

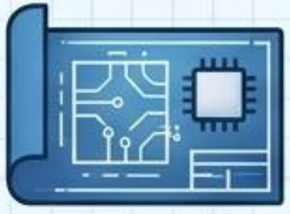
Gemini Notebook

深潛 S7 —— 證據鏈與大腦演化

無證據 = 未完成。逐環點名，缺哪環補哪環。



結案必做三件套（演化與沉澱）



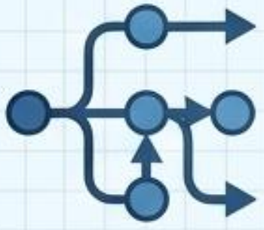
❶ 模式卡入庫

專案中重複出現兩次以上的結構，萃取為 PatternType 供下次複用。



❷ 檢核項沉澱

踩過的坑，寫成「已知誤用」欄位（誤用 ➡ 後果 ➡ 糾正）。

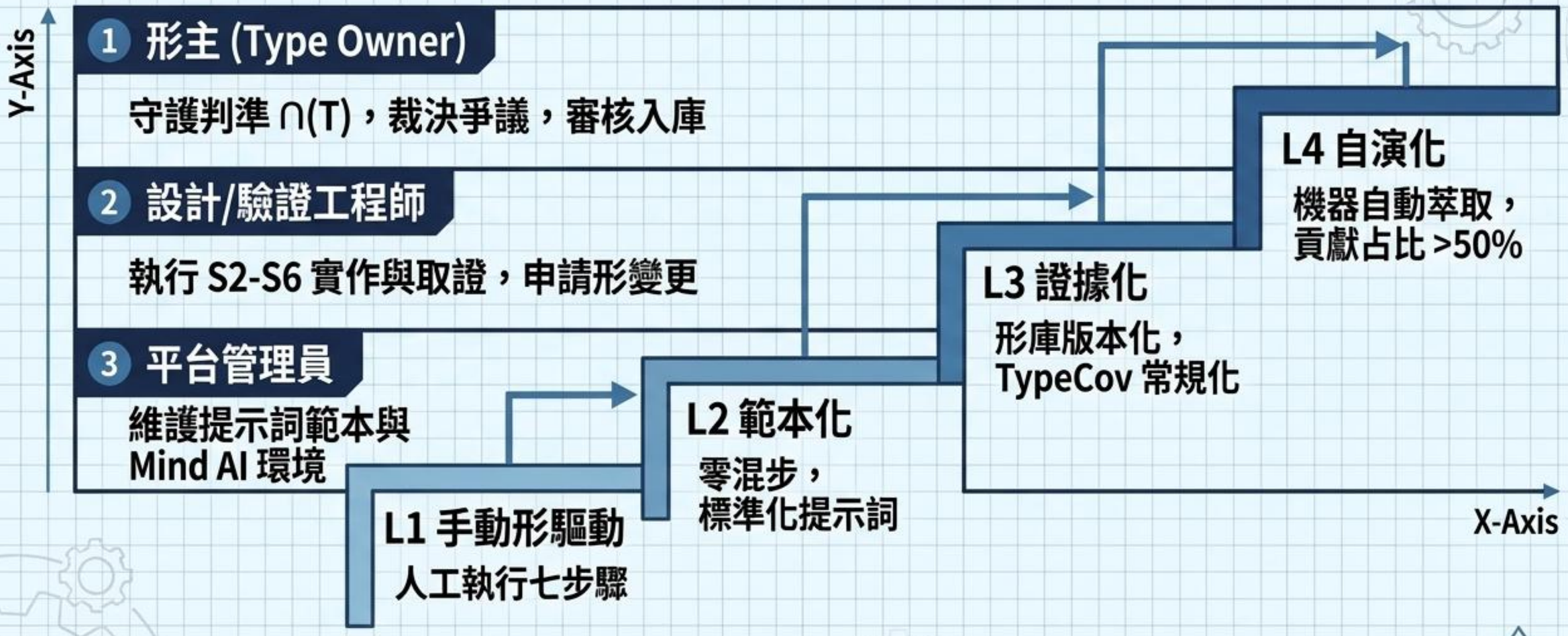


❸ 判準修訂

升級 ⋂(T) 版本。形庫的版本，就是整個團隊的智慧版本。

Gemini Notebook

組織協作 —— 導入路徑與成熟度分級

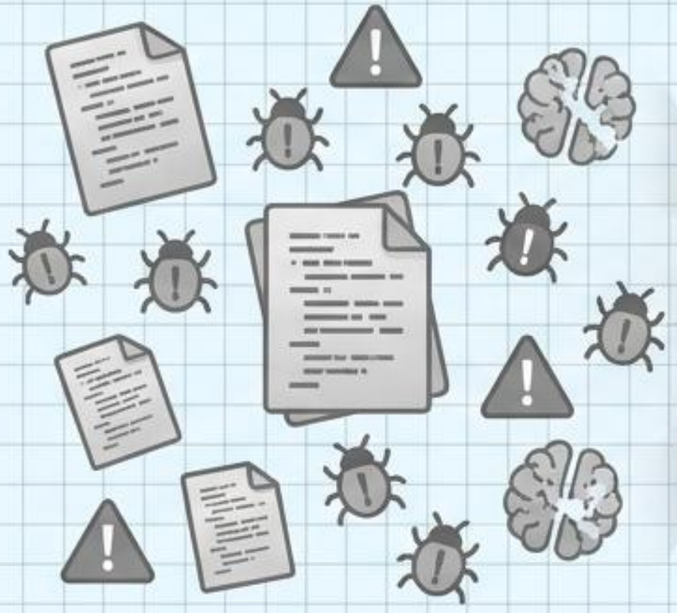


行動建議：選定一個中等複雜度 IP（如 UART），完整走通一遍 S1-S7，勝過三個專案各走半遍。

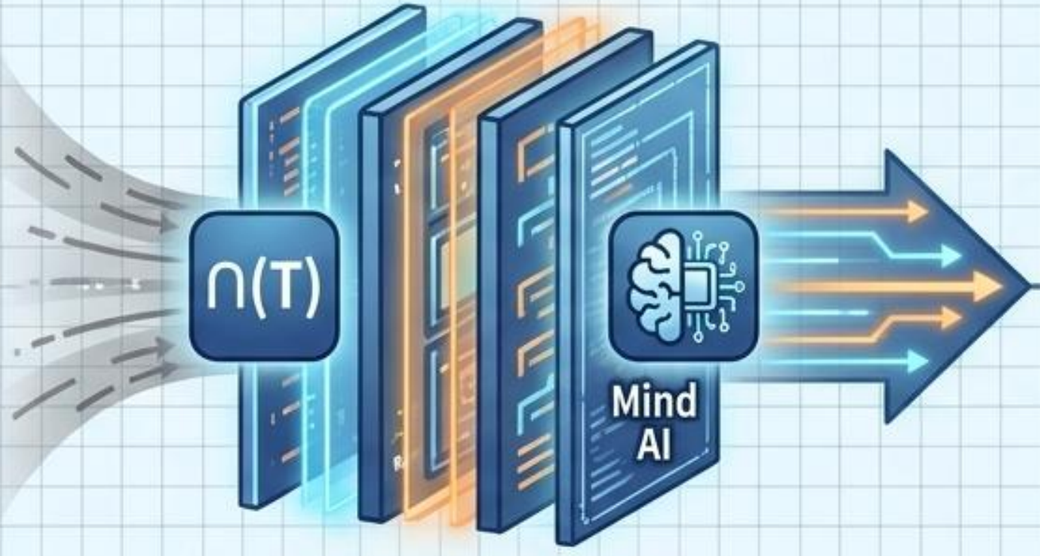
Gemini Notebook

總結合成 —— 讓每次設計都成為智力資產

傳統設計：消耗品



雙本形驅動過濾器 (Dual-Type Filter)



形驅動設計：資產累積



雙本形驅動過濾器 (Dual-Type Filter)

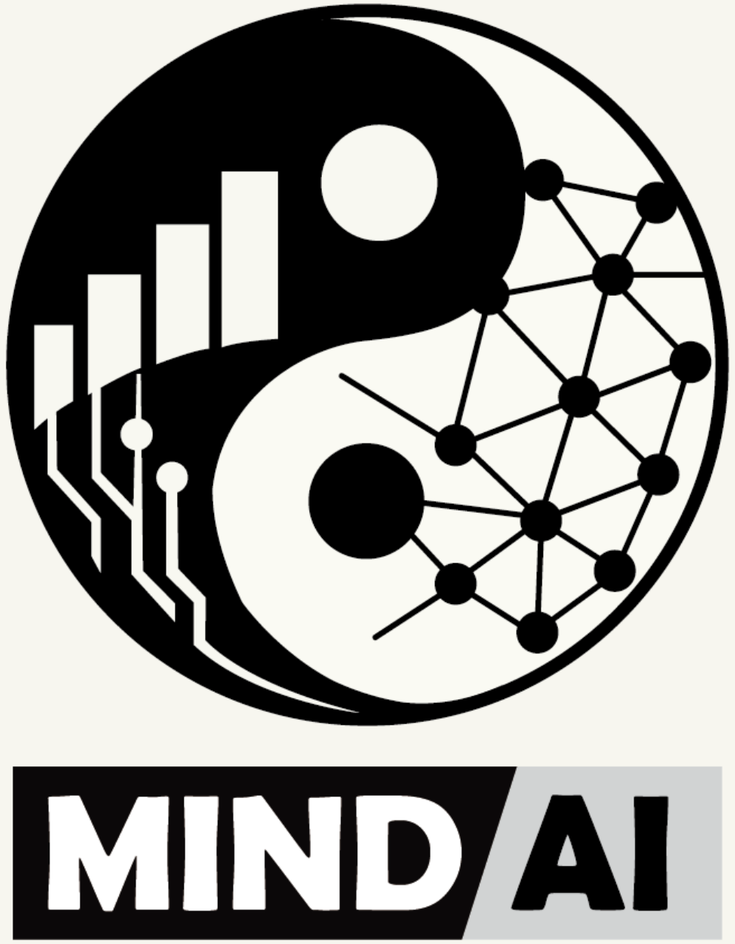
核心洞察 (The Aha Moment)

「準確」不是一種願望，而是四層嚴密的、可稽核的機制。傳統設計做完就歸零；形驅動設計是資產累積——每一次專案衝刺，都在擴張團隊的形庫大腦。

告別感覺與猜測。現在就開啟你的第一個形驅動衝刺 (Type-Driven Sprint)。

Gemini Notebook

LonGreat 願景：創造理解世界本質的智慧生命



「讓人工智慧不只是工具，而是真正具備理解、推理、學習與創造能力的智慧生命體系。」

從知識統一，到思維統一，最終邁向智慧統一。
這就是 LonGreat 所追求的 AGI 永恆之道。

www.longreat.net

張長豐
gamma@livemail.tw

張椁嶺
mindaiworks@gmail.com